Jetson BSP custom agent skills tutorial

1. Description of the function

jetson-bsp-skills is a set of the Jetson Linux BSP customisation workflow Skills provided by NVIDIA. It is aimed at the host side development before the Jetson is flashed, not at the device skills for the daily diagnostics on an already booted Jetson.

The repository does not contain the BSP images, the source code archives or the material of the specific carrier boards; it provides the workflow guidance, the templates and the reference material, and it helps the developer prepare the BSP workspace, modify the BSP, build the related artefacts and then flash and verify them after the confirmation.

Official repository:

```
https://github.com/NVIDIA-AI-IOT/jetson-bsp-skills
```

2. The applicable scenarios

These Skills can be considered when a low level customisation of the Jetson platform is needed, for example:

- Modifying the Pinmux and the GPIO pin multiplexing
- Customising the USB, the PCIe, the UPHY and the MGBE
- Adding or modifying the CSI camera device tree
- Adjusting the clock, the DVFS, the fan policy and the nvpmodel
- Adjusting the reserved memory of the BSP
- Modifying the kernel, the device tree or the drivers and rebuilding
- Integrating the customisation into the BSP image and verifying it after the flashing

If you are only installing an application, deploying OpenClaw, running a model or checking the state of an already booted system, you normally do not need the BSP Skills; you can use the device side tools or jetson-device-skills.

3. The working location and the relation of the directories

The BSP customisation is done in a separate workspace on a Linux host. The host can be a physical Ubuntu computer or an Ubuntu virtual machine that is configured with enough disk space and resources.

An example directory:

```
~/jetson-bsp-workspace/
├── .claude/              # The BSP Skills, the context and the reference material
├── Linux_for_Tegra/      # The prepared BSP flashing directory, determined by the chosen
flow
└── source/               # The kernel, the device tree and the other source code that has
to be initialised when needed
```

Claude Code has to be started from the BSP workspace, not from the directory of the running system of the Jetson. The Linux_for_Tegra here is the BSP working directory to be flashed on the host; it is not the root filesystem that is running on the Jetson.

4. The preparation of the host environment

Prepare the following before you start:

- An Ubuntu Linux host or an Ubuntu virtual machine
- A working network connection and enough disk space
- The BSP material that matches the target Jetson model, the carrier board and the JetPack/L4T version
- Claude Code and a working Anthropic login/authorisation
- If you plan to actually flash: a matching power supply, a USB data cable and a Recovery USB device that can be passed through
- A backup of the important data and a recoverable plan for the original system

A virtual machine can be used for the BSP preparation and the source code work. Before the actual flashing you also have to confirm that the virtual machine can recognise the Jetson Recovery USB and pass it through stably; if the USB device cannot be passed through reliably, do not start the flashing.

5. Get the BSP Skills repository

Run the following in the terminal of the Ubuntu host:

```
git clone https://github.com/NVIDIA-AI-IOT/jetson-bsp-skills.git
cd jetson-bsp-skills
```

If the repository already exists, enter it and check it first instead of cloning it again:

```
cd ~/jetson-bsp-skills
git status
git remote -v
```

6. Create the BSP workspace and install the Skills

Create a separate directory for this BSP work:

```
mkdir -p ~/jetson-bsp-workspace
```

Run the installation script that the repository provides to copy the Skills, the context and the reference material into the workspace:

```
cd ~/jetson-bsp-skills
./setup.sh --workspace ~/jetson-bsp-workspace
```

The installation script creates the following in the workspace:

```
~/jetson-bsp-workspace/.claude/
```

If that directory already exists, the script refuses to overwrite it by default. Check its content first; only consider --force when you definitely want to rebuild it and you have a backup. Do not simply add --force to get rid of the error, because the script deletes the existing .claude/ and then creates it again.

Check the result of the installation:

```
find ~/jetson-bsp-workspace/.claude -maxdepth 3 -type f | sort | head -80
```

7. Start Claude Code in the workspace

```
cd ~/jetson-bsp-workspace
claude
```

On the first start, complete the login or the authorisation following the prompts of Claude Code. Enter the officially recommended entry request in Claude Code:

```
help me setup the bsp customization workspace
```

This request leads into the /jetson-quick-start workflow. Choose the automatic preparation or the guided preparation based on the existing material, following the prompts, and provide the real Jetson model, the carrier board, the BSP/JetPack version and the customisation goal.

Do not just tell the agent "help me DIY a BSP". First state the exact model of the target board, the carrier board model, the current/target JetPack version, the storage medium and the specific hardware requirements.

8. The phases of the BSP workflow

The repository divides the work into the following phases:

8.1 Preparation

Create or select the target configuration according to the device and the BSP version, prepare the BSP inputs, unpack the image, initialise the source code/the toolchain and register the local reference material. The common entry points include:

- /jetson-quick-start
- /jetson-init-target
- /jetson-set-target
- /jetson-download-bsp
- /jetson-init-image
- /jetson-init-source
- /jetson-link-docs
- /jetson-generate-kb

After the preparation is complete, whether the Linux_for_Tegra, the source directory and the knowledge base are generated in the workspace depends on the chosen flow and on the BSP inputs that are provided; do not assume that starting Claude Code automatically generates a complete flashing image.

8.2 Customisation

Call the corresponding /jetson-customize-* Skill according to the requirement, for example the pinmux, the USB, the PCIe, the UPHY, the camera, the clock, the fan or the power mode. When you use a custom carrier board, first confirm whether /jetson-derive-carrier is needed.

Before every modification you should confirm:

- The specific board type and the hardware version that is modified
- The paths of the original file and of the target file
- That the configuration/device tree matches the target BSP version
- The content of the change and its potential impact
- Whether there is a usable original backup and a way of rolling back

8.3 Build and integration

If the modification involves the kernel side source code, use /jetson-build-source to build the DTB, the modules or the kernel artefacts. Then use /jetson-promote-image to integrate the customisation files and the build artefacts into the BSP image to be flashed.

If the change only affects the rootfs or the configuration, the build steps can be different; follow the workflow of the corresponding Skill and of the current BSP version.

8.4 Flashing and verification

Only after the static checks, the diff checks and the target verification are complete, you can consider:

- /jetson-flash-image: flash the prepared BSP onto the Jetson
- /jetson-validate-image: the static checks and the device side verification after the flashing

The flashing may erase the target storage, overwrite the QSPI/the boot components or make the device temporarily unable to boot. Before you start you have to explicitly confirm the target Jetson, the target storage device, the Recovery state, the BSP version and the backup; you cannot conclude from an "empty SSD" that the flashing will not affect the other storage. The flashing medium and whether the QSPI is involved are determined by the target board type and by the corresponding BSP flashing plan.

9. The suggestions for the first experience

When you use it for the first time, do a workflow experience that does not write to the disk:

1. Install the Skills in the Ubuntu workspace.
2. Start Claude Code and run help me setup the bsp customization workspace.
3. Provide the accurate board type and version information.
4. Let the agent explain the Setup flow that it is going to choose and the inputs that it needs.
5. Use /jetson-print-bsp-info to check the existing Linux_for_Tegra directory (if it has already been prepared).

6. First look at the customisation plan and the file diffs; do not execute the flashing.
7. Once you are familiar with the workflow and you have a recoverable environment ready, evaluate the build and the deployment separately.

If you are only trying out the flow, do not connect a Recovery device, do not run /jetson-flash-image and do not let the agent run the commands that erase or overwrite the storage.

10. Frequently asked questions

10.1 setup.sh reports that --workspace is required

The target workspace has to be specified, for example:

```
./setup.sh --workspace ~/jetson-bsp-workspace
```

10.2 It reports .claude/ already exists

The script protects the existing directory by default and does not overwrite it. First check whether there is already work content in it; only use --force to rebuild it after you have confirmed that it can be discarded and backed up.

10.3 Claude Code cannot find the Skills

Make sure that the current directory is the workspace that was specified at the installation and start Claude Code from that directory:

```
cd ~/jetson-bsp-workspace
claude
```

Check whether .claude/skills and the SKILL.md of each Skill exist in the workspace.

10.4 The virtual machine cannot see the Recovery device

First check the USB pass through settings of the virtual machine and the state of the device connection. Confirm in Ubuntu that the Recovery USB device is visible before you plan the flashing. Do not run the flashing flow while the device does not appear in the virtual machine stably.

11. The risk notes

- The BSP Skills provide the workflow guidance; they cannot replace the official NVIDIA documentation, the schematic of the board and the engineering review.
- All the generated plans, commands, file diffs and commit contents have to be reviewed by a human.
- Choosing the wrong board type, the wrong carrier board, the wrong BSP version or the wrong target disk can make the Jetson unable to boot or overwrite the wrong device.
- An error in the device tree, the kernel, the Pinmux or the power configuration can cause the peripherals to fail, the boot to fail or the hardware to behave abnormally.
- The flashing and the overwriting operations are high risk; confirm the target, the backup and the recovery path before you execute them.

- First verify the compatibility of the current Skill and of the BSP version; do not apply the flashing commands of an older tutorial directly to another version.